

[L0-L1 核心本质与设计逻辑]

L0 一句话本质：在多进程隔离与共享内存段的物理界限下，通过 CBO 动态规划搜索最优多表物理 Scan-Join 执行树，借由堆页面 tuple 行头 xmin/xmax 字段实现高并发快照隔离，并通过预写日志与 Auto-vacuum 回绕冻结规避强吞吐崩溃的关系型数据库系统。

L1 四句话逻辑：

- 多进程共享内存架构** (M1) 采用独立的 Backend 后台执行进程隔离客户端，通过共享缓冲区段与 ProcArray 共享全局状态。
- 查询分析重写与规划** (M2) 遍历 Catalog 获取直方图代价，通过 System R 动态规划生成逻辑 Path 并固化为 Volcano 算子执行树。
- Heap 堆页与 Toast 分裂** (M3) 将变长元组通过行指针槽对齐 Heap 页面，超限大字段切片至 Toast 表，B-Tree 索引采用并发右链裂变。
- xmin/xmax 版本与回绕规避** (M4-M5) 在行头封装多版本生命期，依赖 Auto-vacuum 回收老版本并强制 Freeze 以规避 32 位事务 ID 溢出。

[L2 存储引擎数据流演进拓扑]

• [SQL 字符串] → [Postgres 进程分配] → [Parser/Rewrite 规则转换] → [CBO 最优物理 Join 路径选择] → [Shared Buffers 读入 Heap] → [xmin/xmax 读快照过滤] → [WAL 写入与 Checkpoint 刷盘] → [Auto-vacuum 物理垃圾回收]

[M1.1] Postmaster 与 Backend 多进程架构 (Multi-process Architecture)

- 多进程强隔离高隔：**PostgreSQL 采用多进程架构。主进程 Postmaster 监听端口，在客户端连入时调用 fork() 派生一个独立的 Backend 工作进程处理该连接的所有事务。
- 故障单点隔离：**单个连接进程发生内存越界或挂起崩溃时，仅需终止该后台进程，其他连接不受任何物理影响，保证了系统的极高容错性。

[M1.2] 共享内存段物理构成与作用 (Shared Memory Layout)

- 物理共享页表段：**多进程依靠物理共享内存 (Shared Memory) 通信。
- 共享控制区：**共享内存主要划分为 Shared Buffers (共享页面缓冲区)、WAL Buffers (预写日志缓冲区)、全局锁管理器 (Lock Table)、事务状态记录 (Commit Log, CLOG) 以及维护全局进程状态的 ProcArray。

[M1.3] 连接池代理层 pg_bouncer 调优 (Connection Pooling)

- 进程创建高昂开销：**由于 Postgres 每个连接都是 OS 进程，创建和销毁连接需要进行繁重的主页拷贝与系统调用。
- 事务级连接池复用：**生产中必须配置 pg_bouncer。其事务级池 (Transaction Mode) 能在事务结束后立即释放 Backend 给其他客户端复用，规避进程上限导致 CPU 耗尽在上下文切换中。

[M1.4] PostgreSQL 前后端 v3.0 通信协议 (Frontend/Backend Protocol)

- 网络报文规范：**PG 采用 Frontend/Backend 3.0 协议。连接握手时发送 Startup Message，包含库名与认证。
- 两类查询通信：**
 - Simple Query：**发送 SQL 文本，返回 RowDescription 和若干 DataRow，最后以 ReadyForQuery 闭环。
 - Extended Query：**通过 Prepare、Bind、Execute 三阶段报文，支持二进制传输参数。

[M2.1] Parser 词法分析与 System Catalog 检索 (Parsing & Catalog Lookup)

- AST 语法树构建：**Parser 对输入的 SQL 文本进行分词与词法分析，生成抽象语法树 (AST)。
- Catalog 语义校验：**Analyzer 遍历 AST，检索 System Catalog 系统表 (如 pg_class、pg_attribute)，校验表名、列名和数据类型是否存在，输出 Query 结构。

[M2.2] pg_rewrite 规则系统与视图重写打平 (Rules System pg_rewrite)

- 宏展开与替换规则：**Rewriter 接收 Query 结构，利用存储在 pg_rewrite 系统中的规则，将针对视图 (View) 的查询进行扁平化重写。
- RLS 行安全插入：**在此阶段处理行级安全策略 (Row-Level Security, RLS)，动态在 AST 的 Where 条件中追加当前登录用户的筛选子句。

[M2.3] 代价优化器 CBO Join 路径规划 (CBO Join Path Selection)

- 代价公式判定：**CBO (基于代价优化器) 基于单表 Scan 代价，计算生成物理 Path 的代价。
- System R 剪枝算法：**采用 System R 动态规划算法自底向上搜索多表 Join 的连接顺序。通过配置 seq_page_cost 和 random_page_cost 指导优化器在 IndexScan 与 SeqScan 之间做出最优代价决策，自动裁剪低效左深树。

[M2.4] Volcano 算子物理执行节点控制 (Executor Operator Nodes)

- 火山迭代树式调用：**执行器 (Executor) 接收物理 Plan 树。每个算子 (如 HashJoin, SeqScan) 均实现统一接口。
- 自下而上流式拉取：**顶级节点递归调用子算子的 ExecProcNode()。子算子按需流式拉取一行记录 (Tuple) 并返回，将内存占用限制在单行级别，实现了计算流式直通。

[M2.5] Parallel Query 并行执行 Gather 节点 (Parallel Query Workers)

- 并行计算网格分发：**当优化器判断全表扫描代价过高时，会启动 Parallel Plan。
- DSM 内存数据聚合：**主 Backend 进程启动 Gather 算子，派生出多个 Parallel Worker 进程，利用动态共享内存 (DSM) 段分配并行 Scan 页，各 Worker 独立计算并由 Gather 汇总，大幅释放多核 CPU 算力。

[M3.1] Heap Page 堆页面物理结构与槽布局 (Heap Page Layout)

- 槽位指针双向对齐：**PG 默认页大小为 8KB。页面头部包含 PageHeaderData 结构，包含页元数据。
- 堆行数据反向增长：**行的行指针 (ItemId/Line Pointer, 4 字节) 从页头向右顺序增长；实际的 Tuple 元组数据则从页尾向左反向增长。删除元组时只需将对应 ItemId 标为 Unused 并在页面整理 (Page Repair) 时平移元组。

[M3.2] Toast 超限大字段切片表空间技术 (TOAST Spillage)

- 堆页大小物理限制：**由于 8KB 堆页限制，Postgres 规定单行元组大小不得超过约 2KB。
- 附属切片行外存储：**当字段 (如大文本 TEXT) 过大时，触发 TOAST 机制：将数据压缩后切割成若干个不超过 2KB 的 Chunk，物理存放至专属的 Toast 表中，主 Heap 页面仅保留 18 字节的 OID 指针。

[M3.3] nbtpage.c B-Tree 并发右链分裂机制 (B-Tree High-Key & Right-Link)

- 无锁读并发分裂：**Postgres 的 B-Tree 索引页面分裂采用 Lehman & Yao 算法。
- 右链接与 High Key：**分裂时并不立即加锁父节点，而是先在页头部写入 Right-Link 指针指向分裂出的新右页，并记录 High Key。该进程如果发现目标 Key 大于 High Key，可以直接沿 Right-Link 读向右页，避免了全局加锁阻塞。

[M3.4] GIN / GiST / BRIN 多元索引结构应用 (GIN & BRIN Indexes)

- 异构数据查询优化：**
 - GIN (倒排索引)：**针对数组、JSONB 和全文检索，内部为每个元素维护一个 B-Tree 索引项，指向所有包含该元素的行。
 - BRIN (块范围索引)：**针对时间序列等顺序写入大表，仅记录每 128 页 (默认) 的最大最小值，索引大小只有 B-Tree 万分之一，极大减少了 IO 扫描。

[壹] PostgreSQL 并发与物理恢复折衷与异常防范矩阵 (PostgreSQL Concurrency & Recovery Trade-offs)

• \textbf{work_mem 默认大小是最佳的通用配置，不需要根据 SQL 特殊调优}：\textbf{Postgres 的 Sort 和 HashJoin 完全受 work_mem 内存大小限制}。如果排序数据超出 work_mem，会溢出写入磁盘临时文件，导致 I/O 穿透且查询速度断崖下降。→ 用 EXPLAIN ANALYZE 监控 SQL 实际使用排序内存。在需要执行大排序、大数据量 Join 的会话中，临时调大 SET work_mem = '128MB'，使算子在内存中完成。

• \textbf{Auto-vacuum 会占用大量磁盘 I/O 吞吐，应该在写入负载高峰期将其关闭}：\textbf{关闭 Auto-vacuum 会导致大量死元组 (dead tuples) 堆积引发表膨胀}。更致命的是，这会消耗 XID 计数器，一旦超 20 亿限制，Postgres 会为了防止回绕强制只读停机。→ 绝不允许关闭 Auto-vacuum。调优 autovacuum_vacuum_cost_delay 与 limit 参数以平摊其 I/O 负荷，并在生产环境中合理设置 Freeze 阈值，以提前安全回绕事务 ID。

• \textbf{两类查询通信}：\textbf{Postgres 拥有完善的 MVCC 快照隔离，写读写读完全互不冲突，默认可串行化}：\textbf{Postgres 可重复读隔离级别下，Postgres 的 MVCC 无法识别跨表或多对象的因果写倾斜 (Write Skew) 异常，这会导致违反全局一致性约束 (如值班医生均下线)}。→ 在核心防重业务中，显式将事务设置为串行化隔离 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE，使内核自动启用 SSI 谓词锁定，并在回滚时加入自动重试。

